01 - Hybrid - Original

In this notebook, we will try a hybrid (joint) architecture with the original dataset. This model contains:

- Tfidf (as in classical model)
- Bert embeddings (using sentence-transformer library)
- Bert summzarizer model
- Logistic regression classifier

Potentially because of limited training data, automatic doesn't work well with hybrid model on original dataset. This notebook skip the automated hyperparameter tuning process and used our best hyperparameters found during development.

In [1]:
```
# !pip install sentence-transformers
# !pip install bert-extractive-summarizer
```

In [2]:
```
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import precision_recall_fscore_support, f1_score, classificati
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sentence_transformers import SentenceTransformer
from summarizer import Summarizer
```

In [3]:
```
# some main options about using Tfidf, sentence embeddings and summary
train_ratio = 1.0             # the fraction of training set to use. Set to 1.0 for
use_summary = True            # set to False to skip summarization text
embed_model_name = "LaBSe"    # LaBSe model provides good performance and efficienc
concat_vector = True          # combine Tfidf + embeddings. Set to False to use Tfi
use_embeddings_only = False   # set to True to use Bert embedding only, ignoring Tf
```

In [4]:
```
# load dataset
train_fname = "../../data/splits/train_50pc.csv"
dev_fname = "../../data/splits/dev_50pc.csv"

train_df = pd.read_csv(train_fname)
dev_df = pd.read_csv(dev_fname)

print("Train len =", len(train_df))
print("Dev df =", len(dev_df))
```

```
Train len = 83
Dev df = 83
```

In [5]:
```
# in low-resource setting, maybe it's good to use a smaller subset of train data
# to prevent overfitting on train set
train_df = train_df.sample(frac=train_ratio, random_state=42)
```

```
train_df.reset_index(inplace=True)
print(len(train_df))
```

```
83
```

In [6]:
```
train_df.head()
```

Out[6]:

	index	file_id	text	target
0	30	adrso089	Why they're standing on the cookies. And the l...	AD
1	0	adrso021	Everything you see going on in this picture, g...	CN
2	22	adrso046	but it's not real clear so do what you can see...	AD
3	31	adrso270	The mother is drying the dish and the waters r...	CN
4	18	adrso022	And then here on this one. Tell me everything ...	CN

In [7]:
```
dev_df.head()
```

Out[7]:

	file_id	text	target
0	adrso025	Tell me everything that's going on\n\n\n\nthe ...	AD
1	adrso028	She will find her mother worship trying to dis...	AD
2	adrso032	What's going on well the children are climbing...	AD
3	adrso035	Okay here's the picture tell me I'll tell you ...	AD
4	adrso036	Tell me what's going on\n\n\n\nall the action ...	AD

Pre-processing

We will lowercase the text for better efficiency

In [8]:
```
train_df["text_lower"] = train_df["text"].apply(lambda text: text.lower().strip())
dev_df["text_lower"] = dev_df["text"].apply(lambda text: text.lower().strip())
```

In [9]:
```
train_df.head()
```

Out[9]:

	index	file_id	text	target	text_lower
0	30	adrso089	Why they're standing on the cookies. And the l...	AD	why they're standing on the cookies. and the l...
1	0	adrso021	Everything you see going on in this picture, g...	CN	everything you see going on in this picture, g...
2	22	adrso046	but it's not real clear so do what you can see...	AD	but it's not real clear so do what you can see...
3	31	adrso270	The mother is drying the dish and the waters r...	CN	the mother is drying the dish and the waters r...
4	18	adrso022	And then here on this one. Tell me everything ...	CN	and then here on this one. tell me everything ...

In [10]:
```
dev_df.head()
```

Out[10]:

	file_id	text	target	text_lower
0	adrso025	Tell me everything that's going on\n\n\n\nthe ...	AD	tell me everything that's going on\n\n\n\nthe ...
1	adrso028	She will find her mother worship trying to dis...	AD	she will find her mother worship trying to dis...
2	adrso032	What's going on well the children are climbing...	AD	what's going on well the children are climbing...
3	adrso035	Okay here's the picture tell me I'll tell you ...	AD	okay here's the picture tell me i'll tell you ...
4	adrso036	Tell me what's going on\n\n\n\nall the action ...	AD	tell me what's going on\n\n\n\nall the action ...

Apply summarizer model to shorten the text

In [11]:
```
# initialize summarization model
sum_model = Summarizer()
```

In [12]:
```
import warnings
# hide warnings
warnings.filterwarnings('ignore')
```

In [13]:
```
# test text summarization
s = """
"now wait a minute .
everything that's going on in the picture just by looking at it ?
well, she's wiping dishes .
and the boy is gettin(g) cookies out_of a jar .
and he's on a stool .
and there's cups over here .
and the water's going on the floor .
```

```
and the housewife, the mother, is wiping the dishes .
and she's standing by the window .
and &-uh she's lettin(g) the water run out_of the faucet onto the
and the  &+k the ladder is fallin(g) over when he's gettin(g) the
what else you want ?",AD
355-1,"well ‡ the boy is falling .
he's trying to get something  a cookie jar .
and <the stool is>  (.) he's gonna be on the floor in a minute
and &-uh he is tryin(g) to get a cookie for &+h this gal .
and she's over here &-uh +...
oh ‡ &+sh she's not payin(g) any attention .
and she didn't turn off the water .
and the water is drippin(g) all over the floor .
do you see that ?
and she's trying to &-uh +...
=laughs she's in a mess .
she's got <in a>  &+f water all over the floor .
and she's standing in the water .
and &-uh she has an apron on .
and she's tryin(g) to wipe the plates dry .
and &-uh (...) +..."
"""
summary = sum_model(s)
print(summary)
```

```
everything that's going on in the picture just by looking at it ? he's gonna be on t
he floor in a minute
and &-uh he is tryin(g) to get a cookie for &+h this gal . and the water is drippin
(g) all over the floor .
```

```
  File "C:\Users\Minh UBC\miniconda3\Lib\site-packages\joblib\externals\loky\backend
\context.py", line 282, in _count_physical_cores
    raise ValueError(f"found {cpu_count_physical} physical cores < 1")
```

In [14]:
```
if use_summary:
    summ_f = lambda text: f"{text.lower()}. Summary: {sum_model(text).lower()}"
else:
    summ_f = lambda text: f"{text.lower()}" # use it to skip summarization
```

In [15]:
```
# apply summary to dataframes
print("Performing summarization")
print("train set...")

train_df["text_lower"] = train_df["text"].apply(summ_f)

print("dev set...")
dev_df["text_lower"] = dev_df["text"].apply(summ_f)

print("Completed")
```

```
Performing summarization
train set...
dev set...
Completed
```

Build Tfidf Vectorizer

We can use Tfidf Vectorizer to convert text to a fixed-length vector

In [16]:
```
vectorizer = TfidfVectorizer()

corpus = train_df["text_lower"]
print(len(corpus))
print("Building Tfidf Vectorizer...")
X_train_tfidf = vectorizer.fit_transform(corpus)
print("Completed")
```

```
83
Building Tfidf Vectorizer...
Completed
```

In [17]:
```
print(X_train_tfidf.shape)
```

```
(83, 950)
```

Create transformer-based sentence embeddings

In [18]:
```
# initialize sentence embeddings model
vectorizer_embed = SentenceTransformer(embed_model_name)
```

In [19]:
```
corpus = train_df["text_lower"]
print("Creating text embeddings...")
X_train_embed = vectorizer_embed.encode(corpus)
print("Completed")
```

```
Creating text embeddings...
Completed
```

In [20]:
```
print(type(X_train_embed))
print(X_train_embed.shape)
```

```
<class 'numpy.ndarray'>
(83, 768)
```

In [21]:
```
# double check Tfidf vector
print(type(X_train_tfidf))
print(X_train_tfidf.shape)
```

```
<class 'scipy.sparse._csr.csr_matrix'>
(83, 950)
```

Concate 2 vectors (Tfidf + sentence embeddings)

In [22]:
```
# test vector concatenation
v1 = [[2, 4, 6],
      [1, 3, 5]
     ]
v2 = [[8, 10],
      [7, 9]
     ]
v3 = np.concatenate((v1, v2), axis=1) # concate on horizontal axis
print(v3) # should be [2, 4, 6, 8, 10], [1, 3, 5, 7, 9]
```

```
assert np.allclose(v3,np.array([[2, 4, 6, 8, 10], [1, 3, 5, 7, 9]]))
print("Passed")
```

```
[[ 2  4  6  8 10]
 [ 1  3  5  7  9]]
Passed
```

In [23]:
```
def combine_vectors(tfidf_vec, embed_vec):
    """
        combine vectors using global settings (concat_vecor and use_embeddings_only
    """
    if concat_vector:
        result = np.concatenate((tfidf_vec.toarray(), embed_vec), axis=1)
    elif use_embeddings_only:
        result = embed_vec
    else: # use tfidf only
        result = tfidf_vec

    return result
```

In [24]:
```
X_train = combine_vectors(X_train_tfidf, X_train_embed)
print(X_train.shape)
```

```
(83, 1718)
```

Train Logistic regression using combined vector

We can now train a simple logistic regression classifier using processed X_train

In [25]:
```
y_train = train_df["target"]

print("Training logistic classifier...")

clf = LogisticRegression(random_state=42,
                         # class_weight= "balanced",
                         max_iter=200,
                        ).fit(X_train, y_train)

print("Completed")
```

```
Training logistic classifier...
Completed
```

In [26]:
```
# getting score on train set
train_score = clf.score(X_train, y_train)
print(f"Train accuracy = {train_score*100:.2f} %")
```

```
Train accuracy = 91.57 %
```

In [27]:
```
# get precision / recall / f1
y_pred = clf.predict(X_train)
print(classification_report(y_train, y_pred, target_names=["AD", "CN"]))
```

```
                precision    recall  f1-score   support

          AD       0.90      0.96      0.93        47
          CN       0.94      0.86      0.90        36

    accuracy                           0.92        83
   macro avg       0.92      0.91      0.91        83
weighted avg       0.92      0.92      0.92        83
```

Now we can evaluate on dev set

In [28]:
```
X_dev_tfidf = vectorizer.transform(dev_df["text_lower"])
y_dev = dev_df["target"]
print("X_dev shape =", X_dev_tfidf.shape)
print("y_dev len =", len(y_dev))
```

```
X_dev shape = (83, 950)
y_dev len = 83
```

In [29]:
```
# create X_dev embeddings
print("Creating text embeddings...")
X_dev_embed = vectorizer_embed.encode(dev_df["text_lower"])
print("Completed")
```

```
Creating text embeddings...
Completed
```

In [30]:
```
print(type(X_dev_embed))
print(X_dev_embed.shape)
```

```
<class 'numpy.ndarray'>
(83, 768)
```

In [31]:
```
# double check Tfidf vector
print(type(X_dev_tfidf))
print(X_dev_tfidf.shape)
```

```
<class 'scipy.sparse._csr.csr_matrix'>
(83, 950)
```

In [32]:
```
X_dev = combine_vectors(X_dev_tfidf, X_dev_embed)
print(X_dev.shape)
```

```
(83, 1718)
```

In [33]:
```
dev_score = clf.score(X_dev, y_dev)
print(f"Dev accuracy = {dev_score*100:.2f} %")
```

```
Dev accuracy = 77.11 %
```

In [34]:
```
y_pred = clf.predict(X_dev)
print(classification_report(y_dev, y_pred, target_names=["AD", "CN"]))
```

```
                precision    recall  f1-score   support

          AD       0.72      0.85      0.78        40
          CN       0.83      0.70      0.76        43

    accuracy                           0.77        83
   macro avg       0.78      0.77      0.77        83
weighted avg       0.78      0.77      0.77        83
```

Process prediction for test set

In [35]:
```
test_fname = "../../data/splits/test_text.csv"

test_df = pd.read_csv(test_fname)
print("Test df =", len(test_df))
test_df["text_lower"] = test_df["text"].apply(lambda text: text.lower().strip())
test_df.head()
```

```
Test df = 71
```

Out[35]:

	ID	Prediction	text	text_lower
0	adrsdt15	NaN	I see oh boy, almost falling on a stool trying...	i see oh boy, almost falling on a stool trying...
1	adrsdt40	NaN	Well the kids are trying to get the cookies an...	well the kids are trying to get the cookies an...
2	adrsdt26	NaN	Okay, this is a family setting in a kitchen an...	okay, this is a family setting in a kitchen an...
3	adrsdt67	NaN	Oh, what do I see in this picture? This remind...	oh, what do i see in this picture? this remind...
4	adrsdt58	NaN	While the young boy is reaching for a cookie j...	while the young boy is reaching for a cookie j...

In [36]:
```
# add summarization
print("Performing summarization")
print("Test set...")

test_df["text_lower"] = test_df["text"].apply(summ_f)

print("Completed")
test_df.head()
```

```
Performing summarization
Test set...
Completed
```

Out[36]:

	ID	Prediction	text	text_lower
0	adrsdt15	NaN	I see oh boy, almost falling on a stool trying...	i see oh boy, almost falling on a stool trying...
1	adrsdt40	NaN	Well the kids are trying to get the cookies an...	well the kids are trying to get the cookies an...
2	adrsdt26	NaN	Okay, this is a family setting in a kitchen an...	okay, this is a family setting in a kitchen an...
3	adrsdt67	NaN	Oh, what do I see in this picture? This remind...	oh, what do i see in this picture? this remind...
4	adrsdt58	NaN	While the young boy is reaching for a cookie j...	while the young boy is reaching for a cookie j...

In [37]:
```
X_test_tfidf = vectorizer.transform(test_df["text_lower"])
```

In [38]:
```
# create X_test embeddings
print("Creating text embeddings...")
X_test_embed = vectorizer_embed.encode(test_df["text_lower"])
print("Completed")
```

```
Creating text embeddings...
Completed
```

In [39]:
```
X_test = combine_vectors(X_test_tfidf, X_test_embed)

print(X_test.shape)
```

```
(71, 1718)
```

In [40]:
```
test_df["Prediction"] = clf.predict(X_test)
test_df.head()
```

Out[40]:

	ID	Prediction	text	text_lower
0	adrsdt15	CN	I see oh boy, almost falling on a stool trying...	i see oh boy, almost falling on a stool trying...
1	adrsdt40	CN	Well the kids are trying to get the cookies an...	well the kids are trying to get the cookies an...
2	adrsdt26	CN	Okay, this is a family setting in a kitchen an...	okay, this is a family setting in a kitchen an...
3	adrsdt67	AD	Oh, what do I see in this picture? This remind...	oh, what do i see in this picture? this remind...
4	adrsdt58	CN	While the young boy is reaching for a cookie j...	while the young boy is reaching for a cookie j...

In [41]:
```
# convert to 0/1 to be consistent to ground truth
label2binary = lambda label: "0" if label=="0" or label.lower().strip() == "cn" els
assert label2binary("CN") == "0"
```

```
assert label2binary("AD") == "1"
assert label2binary("0") == "0"
assert label2binary("1") == "1"

print("Passed")
```

```
Passed
```

In [42]:
```
# apply to prediction column
test_df["Prediction"] = test_df["Prediction"].apply(label2binary)
# write to prediction output file
output_path = "../../evaluation/"
fname = "test_results_task1_hybrid.csv"
output_df = test_df.drop(columns=["text", "text_lower"])
output_df.head()
```

Out[42]:

	ID	Prediction
0	adrsdt15	0
1	adrsdt40	0
2	adrsdt26	0
3	adrsdt67	1
4	adrsdt58	0

In [43]:
```
output_df.to_csv(output_path + fname,
                 index=False
                )
print("Completed")
```

```
Completed
```

Output from evaluation scripts

$ python evaluate.py task1_gold.csv test_results_task1_hybrid.csv Loading data from task1_gold.csv...done Loading data from test_results_task1_hybrid.csv...done Gold len = 71 Prediction len = 71 Classification report: precision recall f1-score support 0 0.93 0.75 0.83 36 1 0.79 0.94 0.86 35 accuracy 0.85 71 macro avg 0.86 0.85 0.84 71 weighted avg 0.86 0.85 0.84 71 Accuracy = 0.8450704225352113 Recall = 0.9428571428571428